

HACPO: Optimized Specification

This file refines the original HACPO (Hybrid Adaptive Contrastive Policy Optimization) objective: we 1) unify notation, 2) make clipping/PPO mechanics explicit, 3) define the contrastive term and weights, 4) give practical hyperparameter suggestions and 5) provide implementation notes and pseudocode.

Notation (unified).

- θ : policy parameters for $\pi_\theta(y | x)$.
- π_{ref} : reference / behavior policy (fixed when computing ratios).
- $r_i(\theta) = \frac{\pi_\theta(y_i | x_i)}{\pi_{\text{ref}}(y_i | x_i)}$: importance / policy ratio for supervised sample (x_i, y_i) .
- $r_j^{(k)}(\theta) = \frac{\pi_\theta(y_{j,k} | x_j)}{\pi_{\text{ref}}(y_{j,k} | x_j)}$: ratio for the k -th contrastive (negative) sample associated with context x_j .
- A_i : estimated (possibly generalized) advantage for sample i . Use the same notation for all advantages for clarity.
- $w_{j,k} \geq 0$: weight for contrastive sample $y_{j,k}$; we require $\sum_k w_{j,k} = 1$ for each context j .
- Clip interval parameter $\epsilon > 0$; define $\text{clip}^\epsilon(r) = \text{clip}(r, 1 - \epsilon, 1 + \epsilon)$.
- L_{CBO} : contrastive policy optimization loss (defined below).
- $D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta)$: KL divergence used as a constraint/penalty. (See note about direction and sign.)

Objective

We write the HACPO objective in a form consistent with PPO-style clipping and with an explicit contrastive term.

$$\begin{aligned}
 J_{\text{HACBO}}(\theta) = \mathbb{E}_{(x_i, y_i) \sim \mathcal{D}} & \left[\underbrace{\alpha_i \cdot \frac{r_i(\theta) A_i}{\text{clip}^\epsilon(r_i(\theta)) A_i}}_{\text{supervised/PPO-clip term}} \right. \\
 & \left. + \underbrace{(1 - \alpha_i) \cdot \sum_{k=1}^K w_{i,k} \frac{r_i^{(k)}(\theta) A_i^{(k)}}{\text{clip}^\epsilon(r_i^{(k)}(\theta)) A_i^{(k)}}}_{\text{contrastive importance-sampled PPO-clip}} \right] \\
 & + \gamma \mathbb{E}_{x_i \sim \mathcal{D}, \{y_{i,k}\} \sim A} [L_{\text{CBO}}(\theta, x_i, \{y_{i,k}\})] \\
 & - \beta D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta).
 \end{aligned} \tag{1}$$

Notes on signs and KL direction: Many RL formulations either *maximize* an objective (PPO) or *minimize* a loss. Above we write J as an objective to *maximize*; the KL term is written as a *penalty* by subtracting $\beta D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta)$. If you implement using gradient descent on a loss, negate J_{HACBO} accordingly. We use $D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta)$ so that the penalty encourages π_θ to stay close to the reference policy.

Contrastive loss: L_{CBO}

One practical choice for the contrastive policy loss (inspired by InfoNCE-style contrastive objectives, adapted to policies) is:

$$L_{\text{CBO}}(\theta, x, \{y_k\}) = -\log \frac{\exp(s_\theta(x, y_+) / \tau_c)}{\exp(s_\theta(x, y_+) / \tau_c) + \sum_{k=1}^K \exp(s_\theta(x, y_k^-) / \tau_c)}, \tag{2}$$

where:

- y_+ is the positive (target) sample (usually the reference y),

- y_k^- are negative samples drawn from A (or from the current mini-batch).
- $s_\theta(x, y) = \log \pi_\theta(y | x)$ (or any scalar scoring function correlated with log-probability),
- $\tau_c > 0$ is a temperature for the contrastive softmax.

This choice encourages the policy to assign higher relative probability to positive examples than to negatives. Alternatively, you can use margin-based or pairwise losses depending on your task.

Weights $w_{i,k}$ and negative sampling strategy

- **Uniform weights:** $w_{i,k} = 1/K$ is simple and stable.
- **Similarity-based weights:** compute similarity scores (e.g. using s_θ or an auxiliary encoder) and convert to weights via softmax:

$$w_{i,k} = \frac{\exp(\phi(x_i, y_{i,k}))}{\sum_{\ell=1}^K \exp(\phi(x_i, y_{i,\ell}))}.$$

This emphasizes “hard” negatives.

- **Batch negative strategy:** use other examples in the mini-batch as negatives to avoid extra sampling overhead.

Always normalize $w_{i,k}$ so $\sum_k w_{i,k} = 1$ for numerical stability.

Adaptive mixture coefficient α_i

To adapt the relative importance of the supervised vs contrastive terms, define α_i per sample. One robust choice:

$$\alpha_i = \sigma\left(\frac{\ell(x_i, y_i) - \tau}{\beta'}\right), \quad (3)$$

where $\ell(x, y)$ is a sample-level signal (e.g. token length $|y|$, or a difficulty score such as negative log-likelihood under reference policy), τ a threshold, and β' a temperature controlling smoothness. Practical recommendations:

- If $\ell = |y|$ (sequence length), tune τ on a validation set (512 is task dependent).
- Consider alternative signals: model confidence, reference log-prob, or validation loss per example.
- Optionally clip $\alpha_i \in [\alpha_{\min}, \alpha_{\max}]$ to avoid extremes.

You can also schedule τ over training steps (e.g. slowly move from supervised-heavy to contrastive-heavy).

Hyperparameter defaults & practical guidance

- ϵ (PPO clip): common values 0.1^{-1} to 0.5^{-1} ; start with 0.1^{-1} .
- K (negatives per context): 4 to 16 ; prefer batch negatives when K large.
- λ (weight for $L_{\text{contrastive}}$): start with 0.1^{-1} to 0.2^{-1} ; tune to balance supervised vs contrastive objectives.
- β : if used, adaptively adjust as in PPO-penalty variants: if observed KL > target, increase β ; otherwise decrease.
- Optimizer: Adam / AdamW with learning rate tuned separately for policy and for any encoder used by contrastive loss.
- Advantage estimation: use GAE (generalized advantage estimation) to reduce variance.

For the supervised loss, use a standard cross-entropy loss. For the contrastive loss, use the contrastive loss defined above.

Stability considerations and implementation tricks

- **Gradient conflicts:** supervised and contrastive gradient directions can conflict. Consider alternating updates (one step supervised PPO-clip, one step contrastive) or use gradient surgery techniques if conflicts are severe.
- **Clipping and zero gradients:** when ratios are clipped, gradient contributions from those samples are limited. Monitor fraction of clipped updates; if too high, reduce learning rate or increase ϵ .
- **Value normalization:** normalize advantages (zero mean, unit variance) per batch to stabilize scale.
- **Use mini-batch vectorization:** compute ratios and clip operations in a vectorized way for speed.
- **Monitoring:** (1) supervised PPO loss magnitude, (2) fraction clipped, (3) contrastive loss magnitude, (4) supervised PPO loss magnitude. Keep their scales comparable by tuning γ and β .

Pseudocode (sketch)

Algorithm 1: HA-CPO Training Step (sketch)

Require: dataset $\mathcal{D}$, contrastive generator A , batch size B , negatives K

```
1: for each training iteration do
2:   Sample batch  $\{(x_i, y_i)\}_i^B$  from  $\mathcal{D}$ 
3:   For each  $i$ , sample negatives  $\{y_{i,k}\}_k^K \sim A$  (or use batch negatives)
4:   Compute reference probabilities / log-probs under  $\pi_{\text{ref}}$ 
5:   Compute current log-probs under  $\pi_\theta$  and ratios  $r_i, r_i^{(k)}$ 
6:   Compute advantages  $A_i$ , and (optionally)  $A_i^{(k)}$  for negatives
7:   Compute  $\alpha_i$  per Eq.(3)
8:   Compute supervised PPO-clip term and contrastive PPO-clip term (vectorized)
9:   Compute  $L$  per Eq.(2)
10:   $J \leftarrow$  average objective per Eq.(1)
11:  Take gradient step to maximize  $J$  (or minimize  $-J$ )
12:  Update  $\beta$ , adaptively if using KL-penalty scheme
13: end for
```

Summary of improvements over original spec

- Unified notation for ratios and advantages to avoid ambiguity.
- Concrete contrastive loss (InfoNCE-style) option and weight normalization.
- Practical hyperparameter defaults and stability/implementation guidance.

Appendix C: To reduce misinterpretation of implementation links.